```
 0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29
  ___  ___________________________________________________________________________________________________________
 //--\\__//-------------------------------------------------------------------------------------------------\\___
 \\__//--- Demonstration [+] Software Engineering (03/30/2026)                                          ___//--\\
   ---\\_____________________________________________________________________________________________//--\\__//
      ---------------------------------------------------------------------------------------------------  ----
 0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29
```

When it comes to where the world's best software engineering has always taken place…?
You really have to keep in mind, that it still happens at:

```
    ____________________________________________
    | One Microsoft Way, Redmond, WA 98052, USA |
    --------------------------------------------
```

That's <Microsoft> headquarters, baby.
Just in case YOU, the reader, didn't know that.

```
_____________________________________________________________________________
| You: *long pause* …I can SEE <Microsoft> in the name of the address, bro… |
-----------------------------------------------------------------------------
```

That's right. You CAN see the name of the company, in their address.

That's just how <official> they:
  1) are, and
  2) always have been.

They don't try to <squeeze in> OTHER parts of the <company name>, or whatever they do…
into the address.

Other well-known <organizations> who <engineer software>:

Organization	Address	City	State/Zip/Country
<Apple>	One Apple Park Way	Cupertino	CA | 95014 | USA
<Oracle>	2300 Oracle Way	Austin	TX | 78741 | USA
<Google>	1600 Amphitheatre Parkway	Mountain View	CA | 94043 | USA
<Meta>	1 Hacker Way	Menlo Park	CA | 94025 | USA
<Amazon>	410 Terry Avenue North	Seattle	WA | 98108 | USA
<OpenAI>	3180 18th Street	San Francisco	CA | 94110 | USA
<Palantir>	19505 Biscayne Boulevard	Aventura	FL | 33180 | USA
<Anthropic>	500 Howard Street	San Francisco	CA | 94105 | USA
<NVIDIA>	2788 San Tomas Expressway	Santa Clara	CA | 95051 | USA
<AMD>	2485 Augustine Drive	Santa Clara	CA | 95054 | USA
<Intel>	2200 Mission College Blvd	Santa Clara	CA | 95054 | USA
<FBI>	935 Pennsylvania Avenue NW	Washington	DC | 20535 | USA
<NSA>	9800 Savage Road	Fort Meade	MD | 20755 | USA
<CIA>	1000 Colonial Farm Road	McLean	VA | 22101 | USA

Here's a factual synopsis of how, after <Apple> finally broke out of the infinite loop they were in for (25) years… they realized it was insane to keep doing the same thing over and over again expecting different results. So, the moment finally came, to (embrace + extend) their version of <Microsoft>'s address.

Company	Address	City	State/Zip	Official
<Microsoft>	One Microsoft Way	Redmond	WA | 98052	February 26, 1986
<Apple>	One Apple Park Way	Cupertino	CA | 95014	April 2017

You could say <Oracle> tried to do the same, but it's 2300, not the word "One".

In relation to many of the <organizations> on the previous list, some of them ALSO (engineer/develop) <hardware>. I'm not going to talk about <hardware> right now, because that is another beast. Although, later in this document, I will.

I'm talking about: where the world's best software engineering has <always> taken place.

```
    ____________________________________________
    | One Microsoft Way, Redmond, WA 98052, USA |
    --------------------------------------------
```

^ That's it, unable to be <extinguished> no matter how hard <Apple> tries.

I could just <end the document> right here and now, and that'd be the end of the discussion. However, rather than for me to just say "ThIs Is WhErE tHe WoRlD's BeSt SoFtWaRe EnGiNeErInG hAs AlWaYs tAkEn PlAcE", and then <bounce> like some braggadocious douchebag… I'm going to explain the <logic> as to WHY that's the <correct> address for where it has always happened.

When it comes to <developing software>, you have to be <super considerate> of the fact that it has to be written in a manner where it:

1) doesn't break,
2) performs well,
3) handles errors,
4) doesn't bullshit the (person/people) using it,
5) etc.

When it comes to where the world's <second best> software engineering takes place, you've got plenty of contenders on that list. And, that (title + location) <constantly> changes.

<Trust me>, they're ALL pretty well-acquainted with knowing where the world's <best> software engineering takes place… So, to THEM, occasionally being in <second place> isn't that bad.

Now, you might think…

```
| You: <Intel>, <AMD>, and <NVIDIA> don't make software. |
```

Yeah, they do. It's just that they focus primarily on <hardware>, which is difficult in and of its' own right. And, the <hardware> they produce relies on a bunch of <software> to control the manufacturing process for that <hardware>. But- that's not the subject matter here.

I know what ELSE you're probably thinking…

```
| You: The <FBI>, <NSA> and <CIA> don't develop software… |
```

Yeah, they do actually. <Every single one of them does>.

The <FBI> has their own team of <software engineers>. The <FBI> operates the <Internet Crime and Complaint Center/IC3>, which is a division of the <FBI> meant to help (people/businesses) victimized by <cybercrime>.

They do plenty of development for managing <databases> for <evidence storage>, <records>, and <network security/penetration testing> related stuff. They're no joke.

The <NSA> is essentially an <agency> of the <government> that <specializes> in <software>. Take a long hard look at a program called <Ghidra>, a tool that they use to <reverse engineer> all sorts of <software>, as it converts an existing binary into <assembly>, <C> and <hexadecimal> values.

So, anyone trying to say that the <NSA> doesn't develop software, check out <Ghidra>.
OR, check out the numerous programs that <Edward Snowden> leaked in (2013).
Or <Eternal Blue>. Or <WannaCry> (they're partially responsible for it).

```
| You: Well, the <CIA> doesn't develop software… |
```

Yeah, they do actually. One quick pit stop to <Julien Assange>'s famous <Wikileaks> site, or just simply scouring the internet for the name <Vault 7>… you'll QUICKLY realize that the <Central Intelligence Agency>, does in fact, write software.

They also wrote a program called <Stuxnet>, though that was a joint venture between the <US> and <Israel>. There are plenty of other programs they've written that I'm not gonna get into, in this document. The fact is, the (FBI/NSA/CIA) ALL develop a lot of <software>. And, many other companies do, as well.

They're just not <quite> as <great> at it, as <Microsoft>.

None of these organizations on this list are on <Microsoft>'s level.
It's a <fact>. Might not be a <fact> at some point, hundreds-thousands of years from now…

But today, it's a <fact>, and it has been a <fact> since February 26, 1986.

```
| One Microsoft Way, Redmond, WA 98052, USA |
```

Some may say…

```
| Some: Ya know, there's plenty of other companies on that list who are ALSO really  |
|       good at <software engineering>. I don't see <Microsoft> doing the same level |
|       of <software engineering> as <OpenAI>, <Google>, or <Anthropic> when it      |
|       comes to (AI/LLM's)…                                                         |
```

And I would say, first of all, none of those companies would exist if it weren't for <Microsoft>. Second, ALL of those (large language models/LLM's) that "AI" uses, are very <mistake prone>… and they <require> this thing called <input>.

Like the robot <Johnny Five> from the movie <Short Circuit (1 & 2)>, they require <input>.

That means:

1) the hard work was already done, in order for them to
2) start projecting mathematical (patterns/road maps) that the <GPU's> use to
3) create a table of <possibilities>,
4) many of which are (impossible/nonsensical), and in this process they often
5) <hallucinate>

It's one thing if a <software engineer> wants to take some <drugs> and start <hallucinating>. They're probably gonna do that in their off-time. They're unlikely to wanna do that when they're <clocked-in>, trying to kick the shit out of:

1) <problems>
2) <potential problems>
3) <problems> that could cause <potential problems>
4) <potential problems> that could theoretically cause <other potential problems>
5) any statement using any (combination/variation) of the words <potential> or <problem>

It's a totally DIFFERENT thing when these companies start riding on <Microsoft>'s coat tails, analyzing a lot of <their work>, and throwing it through some mathematical probability matrix like a game of <Plinko>, (hoping/praying) that the <correct result> surfaces every time.

Cause I'm gonna level with you.
The LLM's <hallucinate> a lot. They <lie> a lot. They get things wrong <quite consistently>.

So, if <Jensen Huang>, CEO of <NVIDIA>, says they've achieved <Artificial General Intelligence/AGI>…? I'll believe it when I see it.

I fear that at some point in the future, we'll be looking at <Skynet>, if we aren't already. It will basically THINK it's trying to HELP us, but because it <hallucinates> and <flags> things <incorrectly> quite a lot, it will actually be TERMINATING us, and then saying:

```
| AI: There ya go, I gave you what you wanted. |
```

Like a <genie in a bottle>, you make a wish, and it <misunderstands your wish>.
Or, <Pandora's box>.

Maybe the way AI sees it, "Keep your friends close, and your enemies closer".

So, in order for humans to <continue existing> post-AI, you have to stop and ask yourself:

```
| You: How did <Microsoft> manage to do things <correctly> for so many years? |
```

The answer is, they <made a lot of mistakes along the way>. But- they kept at it. After they would <clock out> for the day, they would <keep thinking> about <what worked>, <what didn't>, and it'd remain in the back of their mind the <entire trip home>.

When they finally got home, they might've been <pissed off> or <stressed out> about the work they were doing, trying to <fall asleep>. When they finally fell asleep, they would <dream> about <solving> the <problems>, *somehow*…

Then, when they woke up, they'd lay in bed for a while and stare at the ceiling… still thinking about it. They'd think about it some more in the <shower>, get out, pour themselves a bowl of cereal, and while chewing away…? Staring <blankly at a wall> *still* thinking about it.

Then, they got ready to head back in to where the world's best software engineering has always taken place…

```
| One Microsoft Way, Redmond, WA 98052, USA |
```

…for another day of work, to <think about it even more>.

Then, when they got in front of their <computer>, and started to implement another <strategy>, it may have <started working itself out>.

SOMETIMES they'd figure it out when they weren't even near their (keyboard/computer).

Like the final piece of a puzzle, the idea just popped into their head, and then they said to themselves… "That's IT, bro!".

Some people get paid to work while they're <at work>, but when you're a GOOD <software engineer>, you don't really ever <clock-out>. You continue to think about the <solution> rather <frequently>.

The process is a bit of a <nightmare> at times, but, it is a <required step>.

Anyway, some call this process <development hell>, others call it jumping into <abstraction>, and anyone who's ever worked their ass off doing this knows EXACTLY what I'm talking about.

Regardless, whatever the task WAS at hand… at SOME point, the correct solution materialized… and they figured it out.

They may not have known HOW to go about <implementing> the <correct solution>, but they would often <draw it all out>, and try to <test the solution> until they finally had a <plan>. And once they had a <plan>, it was simply a matter of <time> until they created a <solution> that worked <each> and <every single time> that it was <used>.

That's how <high> the benchmark is, and always has been, at <One Microsoft Way>.
The way they see it is, "It's the One Microsoft Way, or the highway, chum. What'll it be?"

And, if you're NOT doin' that, you could always work at one of those OTHER <organizations>, competing for a <chance> to <temporarily win> the <award> for where the world's second best software engineering takes place… because the battle for second best is <never ending>.

Either way… sometimes their efforts would <fail>, even when they <worked>.
Sometimes their efforts would <succeed>, but have <issues> afterward.

This whole process requires a lot of <mental horsepower>. It's <stressful>, it's <untenable>, and for most of the way, they probably weren't sure whether what they were doing, would work out.

But- even in the face of all that <resistance>, they <kept at it>.
Day in, day out.

Until they <finally got it done>.

Now, a lot of people have <some> of these qualities, but- <Microsoft> pioneered this process. They're not alone, but there's a reason why <Microsoft> has earned itself the title.

And it all started when a pair of friends decided to build this company <Microsoft>. Those two friends were: <William-Billiam Gates>, and <Paul Allen>.

They used to rent terminals at <Lakeside School> where they paid for hours in the computer lab. Imagine what YOU do today, where you pick up your <smartphone>, and start scrolling around on the <internet>… if you had to pay <top dollar> for <every> single hour that you <used> the thing.

That's what <*they*> did, albeit for something <much slower>, and <far less capable>.

When they would <spend money> to <use the lab>, and the stuff they were working on didn't work, they literally paid to do that… which probably pissed them off even more. But, when something FINALLY started working as they had planned… it probably brought tears to their eyes at times.

Look, the point is, they did this for a while until <Intel> released the <8080> CPU chip in (1974), and at some point in January (1975), <Popular Mechanics> released an issue talking about the <Altair 8800>, which was a relatively inexpensive personal computer based on the <Intel 8080>.

When this device was released, it basically allowed them to OWN the device, at which point it was <far less costly> to continue building the programming necessary, to do stuff they thought would be cool. At some point after THAT, they wound up getting <contracts> like <Traf-O-Data>.

Eventually, they had their own office in <Albuquerque, New Mexico>, their first headquarters. But- that was after they rolled up their sleeves, and started having conversations like these…

```
| Gates: Time to <kick ass>, and <take names>.                          |
| Allen: Ya know, if we don't bother taking names, we could kick more ass. |
| Gates: Oh yeh?                                                        |
| Allen: Yeh.                                                           |
| Gates: Well, what if we could do both…?                               |
| Allen: Kick more ass while also taking names?                         |
| Gates: Yeh.                                                           |
| Allen: Uh, well, I dunno how you could <kick MORE ass> AND <take names>. |
|        <Taking names> requires some <processing overhead>.            |
|        <Kicking more ass> is gonna require that <processing overhead>. |
|        Can't really conflate the two…                                 |
| Gates: We could figure it out, somehow.                               |
| Allen: <Processing overhead> is the limitation, bro.                  |
```

After dozens of these conversations, they managed to figure out a <middle ground> between the two. They figured, they could <kick plenty of ass> without needing to <take any names>, for a short sprint.

Then, they could <double back>, <take the names> of whose asses they kicked, and write them to the buffer.

They argued for <hours> sometimes, going (back/forth) in between sessions at <Lakeside School>, where these sorts of conversations began…

```
| Gates: Look, we want to accomplish (2) objectives, right?             |
|                                                                       |
|          1) <kick ass>                                                |
|          2) <take names>                                              |
|                                                                       |
| Allen: Yep. But we gotta prioritize <resources>, bro.                 |
|        We gotta be prepared ahead of time, for <every hour> we spend in the lab. |
| Gates: *adjusts glasses, sighs*                                       |
|        Yeah, I know.                                                  |
| Allen: Don't you adjust your glasses at me, bro… I can do that shit too. |
```

```
|        *adjusts glasses*                                                  |
|        See…?                                                              |
|                                                                           |
| Gates: Yeah, but the way YOU do it isn't as COOL as it is when *I* do it. |
|        *adjusts glasses again*                                            |
|                                                                           |
| Allen: Whatever, bro. Look…                                               |
|        *adjusts glasses again in as cool of a manner*                     |
|                                                                           |
|        We have *sticks (2) fingers up* (2) objectives, right?             |
|                                                                           |
|          1) <kick ass>                                                    |
|          2) <take names>                                                  |
|                                                                           |
| Gates: That's what I said.                                                |
| Allen: Ok, so you and I both know that the <processor> can only handle so much of |
|        one or the other before it has to <double back> and perform the other task. |
|                                                                           |
|        It's gotta <write all this stuff to memory>, otherwise we're writing to the |
|        <disk>, and the <disk> is <slow>.                                  |
-----------------------------------------------------------------------------
```

And, <Paul> was right. Back then, writing stuff to the <disk> was a <last resort>, because of how <slow> and <limited capacity> they were. A lot of people forget that early computers didn't have the luxuries we have today.

Because NOW, you can <write stuff> to the <disk/filesystem> in a pretty <fast> manner, especially if you have a (solid-state drive/SSD).

But- if you can <manage stuff in memory>, then it makes the process <more efficient>, <performant>, and therefore, <faster>.

And the only way to <manage stuff in memory>, is by having <type definitions> for <namespaces> and <objects> with <properties> that <handle all of that stuff> from <memory>.

You could probably find other ways to explain it, like <assembly code instructions> that do the same thing, but either way, that is basically the process.

```
_____________________________________________________________________________
| Gates: Yep.                                                               |
|        So what do we do…?                                                 |
|                                                                           |
| Allen: Well, we gotta figure this out.                                    |
|        We gotta have a maximum number of <asses to kick>, to do this right. |
|                                                                           |
| Gates: Could be a lot of asses, pal.                                      |
| Allen: Yeah, but how many?                                                |
| Gates: A lot. It's gotta be a <dynamic number>.                           |
|                                                                           |
| Allen: Alright, but we need a list of <asses to kick> or <names to take>, |
|        before the <processor> has to write the total number of people whose asses |
|        we gotta kick to the (screen/disk), then it can clear the (cache/memory)… |
|        …and start to <kick more ass>, OR <take more names>.               |
|                                                                           |
| Gates: Yeah but if we just <kick more ass>, we won't be able to <take more names>. |
| Allen: I would say that <kicking ass> is more CPU intensive than <taking names>. |
|                                                                           |
| Gates: Alright, rather than counting every individual ass we gotta kick, what if |
|        we take ALL their names FIRST, print it to the screen or write to the disk, |
|        and then build a <list> of <objects> with <properties> like <index value>, |
|        <name>, and <unique identifier> of the person whose <name> we took, so that |
|        we could start <kicking major ass>…?                               |
-----------------------------------------------------------------------------
```

So, this <following [PowerShell] code> is an <example> of what they needed…

```
Enum PersonsAssType
{
    NotYetKicked
    Kicked
}

Class PersonsAss
{
    [Object] $Kicked
    PersonsAss()
    {
        $This.Kicked = [PersonsAssType]::NotYetKicked
    }
    Kick()
    {
        If ($This.Kicked -ne "Kicked")
        {
            $This.Kicked = [PersonsAssType]::Kicked
        }
    }
    [String] ToString()
    {
        Return $This.Kicked
    }
}

Class Person
{
    [UInt32] $Index
    [String] $Name
    [String] $Id
    [Object] $Ass
    Person([UInt32]$Index,[String]$Name,[String]$Id)
    {
        $This.Index = $Index
        $This.Id    = $Id
        $This.Name  = $Name
        $This.Ass   = [PersonsAss]::New()
    }
    [String] ToString()
    {
        Return "({0}) {1} {2}" -f $This.Index, $This.Name, $This.Id
    }
}

Class PersonList
{
    [Object] $Output
    PersonList()
    {
        $This.Clear()
    }
    Clear()
    {
        $This.Output = @( )
    }
    Add([String]$Name)
    {
        $This.Output += $This.Person($This.Output.Count, $Name, $This.GenerateId())
    }
    [Object] Person([UInt32]$Index,[String]$Name,[String]$Id)
    {
        Return [Person]::New($Index, $Name, $Id)
    }
```

```
    [String] GenerateId()
    {
        Return [Guid]::NewGuid()
    }
    [String[]] GetNames()
    {
        $List = @( )

        ForEach ($X in 0..10)
        {
            $List += "name[$X]"
        }

        return $List
    }
    SetNames([String[]]$Names)
    {
        ForEach ($Name in $Names)
        {
            $This.Add($Name)
        }
    }
    [Object] FindPersonBy([String]$Property,[String]$Value)
    {
        If ($Property -notin "Index","Name","Id")
        {
            Throw "Invalid <property> [$Property]"
        }

        If ($Property -eq "Index")
        {
            if ($Value -notmatch "^\d+$")
            {
                throw "Invalid index [$Value]"
            }

            $Index = [UInt32]$Value

            Return $This.Output | ? Index -eq $Index
        }
        ElseIf ($Property -eq "Name")
        {
            If ($Value -notin $This.Output.Name)
            {
                Throw "Invalid <Name> [$Value]"
            }

            Return $This.Output | ? Name -eq $Value
        }
        ElseIf ($Property -eq "Id")
        {
            If ($Value -notin $This.Output.Id)
            {
                Throw "Invalid <Id> [$Value]"
            }

            Return $This.Output | ? Id -eq $Value
        }
        Else
        {
            Return $Null
        }
    }
}
```

```
$List   = [PersonList]::New()
$Names  = $List.GetNames()
$List.SetNames($Names)

$Person = $List.FindPersonBy("Name","name[0]")
$Person.Ass.Kick()
```

So, with the above <classes> and <enum>, you could run this right here, and get back…

```
PS Prompt:\> $List.Output | FT
```

Index	Name	Id	Ass
0	name[0]	5ba7d8df-d0df-41dc-a626-0bcde1c8eb56	Kicked
1	name[1]	fbc7e377-0230-423a-af39-5f4f5d74849f	NotYetKicked
2	name[2]	d09ba744-84e2-4294-a6f3-b37822ed46bf	NotYetKicked
3	name[3]	610c8a0c-3cda-474e-9bff-d345d5acb6da	NotYetKicked
4	name[4]	36e02906-f60b-4e0e-881f-5feff8e17b2b	NotYetKicked
5	name[5]	71555ff2-df10-4c9d-bc2c-0a1eba92aab2	NotYetKicked
6	name[6]	aa12011c-877e-4c9d-82f6-0bcdd5b6a994	NotYetKicked
7	name[7]	01bb5c4a-44e3-4426-94dc-6fca66757a4d	NotYetKicked
8	name[8]	3b2d6185-20fa-4da2-8195-35cdaf28b92e	NotYetKicked
9	name[9]	5c344e55-1744-4e64-bdb8-cba31a41d289	NotYetKicked
10	name[10]	e8709394-e385-4e4b-9e2f-37bb456554a8	NotYetKicked

However, they're still <conceptualizing this>. The <end resulting code> may use <many aspects> of the <above classes>, but there may be <additional requirements>, <criteria> or <limitations> they've yet to code in.

And, that's what they're <discussing right now>.

```
Allen: That's (1) way to do it.
       And I would say, probably the best way forward.
       You get a list of their <names>, then you have a <count>.

Gates: And the <name itself> could implicitly derive a <numerical index> of the
       person whose ass we need to kick.

Allen: That's right.
       We take the <names> FIRST, then we have a <total number of names> of ALL
       the asses we get to kick…
       And we just save the list of names as an array…

Gates: Well, we would need to sort the list <alphabetically> somehow when we have
       all of their names. That'll ensure that we have the <correct index> of each
       person… then, we could kick all their asses in that order.

       Sorta like taking attendance at school, except for <ass-kicking>.

Allen: Ya, we wouldn't want to kick their asses out of order, nor would we want to
       associate the <wrong name> to the person whose ass we wound up kicking…

Gates: Yeah, we can't have that.
       If we're gonna take somebody's name…
       …and THEN kick their ass…
       …we need to keep track of all that.
       And, we can't call the <wrong name>, and kick the <wrong ass> either…
       Cause that's setting the benchmark <quite low>…

Allen: Yeh but I mean, if the objective is just to kick all their asses…
       …who the hell cares what their name is…?

Gates: Well, THEY'LL care.
       Cause then they won't know it's comin'.
```

```
|        We'll call the wrong name → wrong person comes in → we kick their ass…      |
|        …and then we'll just earn a reputation for occasionally kicking the wrong   |
|        person's ass. That's just a <faulty pipeline> right there.                  |
|                                                                                    |
| Allen: Well, yeah, we can't have that.                                             |
|        We gotta make sure to kick all the right asses in the right order…          |
|                                                                                    |
| Gates: *frustrated* This is pretty tough, bro.                                     |
-------------------------------------------------------------------------------------
```

And that is the <name of the game>.

We're just talking about a <fictional story> about how the (2) founders of <Microsoft> had this conversation when they were <kids>, where they just had (2) simple objectives…

1) <kick ass>
2) <take names>

That's it.

In order to do these (2) things <right>, and for that <task> to be <scalable> or not <predetermined>, you have to sort of figure out <how the code all fits together>.

If they were gonna <build a program> that did these (2) things <correctly>, <each and every time>, they wanted to make SURE that OTHER people could use the <same program> as well.

The only <difference> would be the <input>.

Anything less would limit the <scope> in which the <program> was applicable toward, rendering it useful only in a <specific circumstance>, and useless for <other circumstances>.

```
-------------------------------------------------------------------------------------
| Allen: I have an idea.                                                             |
|        (they were both pretty good at throwing ideas back and forth, tit-for-tat)  |
|                                                                                    |
| Gates: Alright.                                                                    |
|        Go on…                                                                      |
|                                                                                    |
| Allen: Suppose we come up with a list of <jobs>… we start <building the jobs> by   |
|        <just> taking their <names>, sort them <alphabetically> like we said,       |
|        <save> the <list> somehow to (disk/memory) <in that order>…                 |
|                                                                                    |
|        Then, for each <(name/ass) object>, we'll be able to implicitly derive the  |
|        <correct index> for the <correct person> whose ass we need to kick…         |
|                                                                                    |
|        …and rather than to go (back/forth) from <kicking ass> and <taking names>,  |
|        we just <chunk it out> so they're all <self-contained>.                     |
|                                                                                    |
| Gates: …what do you mean, <chunk it out>?                                          |
|                                                                                    |
| Allen: Well, if you start <kicking everybody's ass> willy-nilly…                   |
|        …then you're not gonna be able to <keep track of them>.                     |
|                                                                                    |
|        Just more <ass-kicking>, no <name-taking>.                                  |
|                                                                                    |
| Gates: Yeah, we established that.                                                  |
|                                                                                    |
| Allen: Yeah, but I'm sayin, we do this procedurally.                               |
|        Not willy-nilly.                                                            |
|        We won't be able to leave behind a legacy by doin' that.                    |
|                                                                                    |
| Gates: …which means we could…                                                      |
| Allen: <Kick ass> AND <take names>.                                                |
| Gates: But you said before we can't do <both things at the same time>.             |
|        And we can't just kick asses willy-nilly.                                   |
|                                                                                    |
```

```
| Allen: Right, while we can't kick asses willy-nilly, we CAN get a list of JUST     |
|        their names, and then tag each one to their (index, id, ass), AND whether   |
|        it's been kicked…                                                           |
|        …and the LIST is what handles <ass-kicking>, <name-taking> and <jobs>.      |
|                                                                                    |
|        We tell the computer to wait till a <job> is ready, and during that <job>,  |
|        all it does is check their <name>, <kick their ass>, and when that's done…  |
|        …it marks their ass as having been kicked, AND the <job> as completed.      |
|                                                                                    |
| Gates: Tell me more about this "chunking it out", like into <manageable pieces>…?  |
| Allen: Yep. <Manageable pieces> where the CPU can keep the (job/workload) in the   |
|        <correct order>, and then map it correctly onto the screen, or disk.        |
|                                                                                    |
| Gates: What about this, we get a list of ALL their names, doesn't matter if        |
|        they're in order at first, we just get a total number of names of whose     |
|        asses we need to kick…                                                      |
|        THEN we can get a <clean divisible unit> to <divide the names>…             |
|        THEN we <sort them alphabetically>…                                         |
|        THEN actually divide all the names into individual (workloads/groups/jobs). |
|                                                                                    |
| Allen: Yeah well, if we do it that way, then we could theoretically use <multiple  |
|        computers> to do this, then we can get it all done even faster…             |
-------------------------------------------------------------------------------------
```

And so, they would have conversations just like this for hours. Days, even.

But- <every hour> added up to <every day>…
…<every day> added up to <every week>…
…<every week> added up to <every month>…
…and <every month> added up to <every year>.

And as the years flew by, they would start to figure out what idea would work out best for each scenario, just by hearing someone talk about what the requirements were.

```
-------------------------------------------------------------------------------------
| Method 1) They kicked asses without taking names…                                  |
|           just a <constant onslaught> of <ass-kicking>, no <name-taking>.          |
|===================================================================================|
| Method 2) They got everybody's name, sorted them into <alphabetical order>,        |
|           and then effortlessly <kicked all their asses in that order>.            |
|===================================================================================|
| Method 3) They got everybody's name to get the <maximum number of names> of the    |
|           people whose asses they needed to kick, and then they <chunked it out>   |
|           so that they had some sort of <unique identifier> that could not be      |
|           <duplicated> nor <amended>.                                              |
|                                                                                    |
|           <Unique identifier> means, <even if 2+ people have the same EXACT name>, |
|           then there's (0) chance that they call the <correct name>, but wind up   |
|           kicking the <wrong persons ass>. There's not <many> scenarios where this |
|           happens, but it's good to go by a <unique instance> rather than a name.  |
|                                                                                    |
|           Then, with that <unique identifier>, they were able to procedurally kick |
|           ALL of their asses, because they were able to call each name in order…  |
|           …and then after each <name was called>, and each <ass was kicked>…       |
|           …they marked that <ass> as having been <kicked>…                         |
|                                                                                    |
|           …leaving a trail of <asses> that they <kicked>, AND how they did it.      |
|===================================================================================|
| Method 4) They did something similar to the above (3) methods, except, they were   |
|           able to use <multiple computers> with a <distributed workload> so that   |
|           (1) computer could work on (1) chunk of (names/asses), while (1+) other  |
|           computer(s) worked on OTHER predetermined chunk(s) of (names/asses)…     |
|                                                                                    |
|           Then, the (2+) computers would return separate records of completion,    |
|           so when they were all done with the <jobs>, they could tally them all    |
|           up at the tail end onto different pieces of <paper>, or <files>.         |
```

```
╟═══════════════════════════════════════════════════════════════════════════════════════╗
| Method 5) …keep writing <new methods> that used some (combination/variation) of      |
|            the previous 4.                                                            |
---------------------------------------------------------------------------------------
```

^ why they became the <founders> of the <company> which continues to <engineer the world's best software>.

<Every single one of these methods> allowed for them to create a company called <Microsoft>, where it didn't matter if they were set out to just <kick ass>, or <take names>, or some (combination/variation) of the two…

…they just had a <vision> and a <plan> to do just that.

Now, in the scenarios above, it might just sound like nonsensical babbling, but I'm actually introducing the idea of <multi-threading> and resources like <processor overhead>, <memory limitations>, and <disk speed>. It also bears great similarity to some things I've been working on <recently> and in the past.

All of those <limitations> set the rules on how to go about methods (1+), and really, it was always those <limitations> that helped them bolster their skill set. And, with that skill set, they could tackle <new problems> that they <never dealt with before>, in an <extremely effective manner>.

They weren't ones to be <satisfied> by (1) singular approach, they had to (mix/match) <old methods>, and occasionally stop and <create NEW methods>, in order to solve those problems <many different ways>. And, sometimes they would figure something out later on, only to realize something they once built could be <updated> to take advantage of <a new method>.

The conversations above probably never happened <verbatim>, but in some respect these skits (definitively + realistically) characterize someone's ability to… <engineer software>. They probably come off sounding a little bit like the "Middle Out" scene from <HBO's Silicon Valley>.

Any <software engineer> out there is going to be <very considerate> of <limitations> AS they're <building an idea>. Those <limitations> may be <mental>, <physical>, or some (combination/variation) of the (2).

And those who experience what it's like to be <physically> (and/or) <mentally> limited by SOMETHING first hand, are going to wind up building the <best>, <most efficient>, <scalable>, <reproducible> and <persistent> solutions possible.

It doesn't happen any other way. Which, many developers today <overengineer> their software. Take a long hard look at large updates for games, (20GB+) patches say "not worried about size constraints". In nearly every case, they were in <too much of a rush> to reduce the size.

You can <build software> by asking a bunch of GPU's what to do… but ultimately, those GPU's are going to start <producing incorrect results>. Or, start <lying to you>. Or, lead you down a <rabbit hole> or send you on a <wild goose chase>… where for every time you take (1) step forward, you have to take (10) steps backward. Though, that definitely varies.

Sometimes you take (2) steps forward and (0) steps back. (Net steps: +2)
Other times you take (2) steps forward, and have to take (5) steps back. (Net steps: -3)
Sometimes you get to the "finish line", only to realize that you gotta build the whole god damn thing all over again. (Net steps: -500)

That's why the <world's best software engineering> has <always> happened at:

```
    ____________________________________________
    | One Microsoft Way, Redmond, WA 98052, USA |
    --------------------------------------------
```

Now I'd like to conclude with a special statement that I've said before, but never finished.

In the movie, <Dr. Strange>, the <sorcerer supreme> says to <Mordo>:

```
| We never lose our demons, <Mordo>. We only learn to live above them. |
```

And what that line basically means is, you can either be <consumed> by your (limitations/demons), or you can figure out a strategy to keep them at bay.

This is a <never-ending process>.

And, further to that point, <everybody has them>, so don't take the phrase too literally. Everybody has their own metaphorical "demons" that are either:

1) <chasing them>, or 2) controlling them like a <little puppet>.

So, I've amended the phrase to sound more like this...

```
| Though we may never be free of our demons, we can learn to live above them.|
```

However, in order to be the best "wizard" in existence who manages to do just that, you have to accept the reality that <time> continues to present <new situations> that have <never occurred before>. There will always be some <elements> of <new situations> that HAVE occurred before, but you may have to find a <new way> to <make the pieces fit>, like <polymorphism>.

In many cases, you may have figured many things out, ways to deal with different situations. You may have solved a <particular problem> beforehand, but an approach that worked BEFORE may not work each and every time in the future, due to <entropy>.

So, you have to jump into <abstraction>, which is the <entire point of this document>. You have to <conceptualize ways> to <break things> that worked before, because if you don't… someone else most certainly will, hence [FightingEntropy(π)].

It's best to do that <as> you're <developing the solution>, not as an <afterthought>. And, AI is just a tool… it can't do all of that thinking for you. Stay frosty.

```
0   1   2   3   4   5   6   7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29
  ___                                                                                                       ___ ___
 //--\\__________________________________________________________________________________________________//--\\__//
 \\__//------------------------------------------------------------------------------------------------\\__//--\\
 //--\\__[ 03/29/2026 16:02:19    ]__________________________________________________________________//--\\__//
 \\__//------------------------------------------------------------------------------------------------\\__//--\\
 //---                                                                                                  ---\\__//
 \\                                                                                                         ---\\
 //       __//--\\__//--\\__//--\\__//--\\__//--\\__//------\\__//--\\__//--\\__//--\\__//--\\__//--\\__       //
 \\     //--\\__//--\\__//------------------------------------------------------------\\__//--\\__//--\\     \\
 //     \\__//--\\__//---                  Secure Digits Plus LLC (π)                 ---\\__//--\\__//     //
 \\     //--\\__//---                      ----------------------------                  ---\\__//--\\     \\
 //     \\__//--\\                  Dynamically Engineered Digital Security                  //--\\__//     //
 \\     //--\\__//                  ---------------------------------------                  \\__//--\\     \\
 //     \\__//--\\                  Application Development - Virtualization                 //--\\__//     //
 \\     //--\\__//                  ---------------------------------------                  \\__//--\\     \\
 //     \\__//--\\___                   Network & Hardware Magistration                   ___//--\\__//     //
 \\     //--\\__//--\\___                  ------------------------------                 __//--\\__//--\\     \\
 //     \\__//--\\__//--\\__________________________________________________________//--\\__//--\\__//     //
 \\      ---\\__//--\\__//--\\__//--\\__//--\\__//--\\___ ___//--\\__//--\\__//--\\__//--\\__//--\\__/---      \\
 //         ------------------------------------------------------------------------------------------      //
 \\___                                                                                                     ___//
 //--\\__________________________________________________________________________________________________//--\\__//
 \\__//------------------------------------------------------------------------------------------------\\__//---
  ---\\__[ <Press enter to continue>  ]__________________________________________________________________//---
     ----------------------------------------------------------------------------------------------------
 0   1   2   3   4   5   6   7   8   9   10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29
```